

Introdução aos Shaders

Por [AV8](#), Agosto 2026

<https://av8visual.vercel.app/>

Este documento é uma introdução ao mundo dos shaders. Seu objetivo é apresentar os conceitos fundamentais utilizando uma linguagem simples e acessível.

Embora vejamos alguns trechos de código, o interesse principal não está na programação, mas sim em compreender a lógica dos shaders: o que são, como funcionam e quais são as ideias principais por trás deles. Para quem deseja se aprofundar nos aspectos de programação, ao longo do documento são incluídas algumas referências e recursos adicionais.

Recomenda-se ler o documento em ordem, já que os conceitos são apresentados de forma progressiva e muitos deles servirão de base para compreender as seções seguintes.

- [Uma breve história dos shaders p. 1](#)
- [Como funcionam os shaders? p. 4](#)
- [ISF \(Interactive Shader Format\) p. 7](#)
- [Vertex Shader p. 8](#)
- [Fragment Shader p. 13](#)
- [Referências p. 20](#)

Uma breve história dos shaders

Para entender o que são os shaders, primeiro convém diferenciar duas ideias distintas.

Por um lado, o **shading** (sombreamento) como conceito, a pergunta de como calcular a cor e/ou a iluminação de uma superfície na computação gráfica. Por outro, o **shader** como um programa que é executado na GPU para gerar um resultado visual, imagem ou animação. Conceito e programa.

É importante fazer essa diferenciação porque o conceito de *shading* surgiu muito antes dos shaders programáveis que conhecemos hoje.

Durante os anos 60 e início dos anos 70, os gráficos por computador eram formados principalmente por modelos em linhas, tipo *wireframe*. Não existiam materiais, sombras ou iluminação. À medida que a computação gráfica começou a ser utilizada em pesquisa, engenharia e, posteriormente, na indústria do cinema, surgiu a necessidade de representar os objetos com uma aparência mais convincente.

Como resposta surgiram os primeiros algoritmos de sombreamento, entre eles o **Flat Shading**, **Gouraud Shading** e **Phong Shading**. Esses algoritmos permitiam calcular a cor de uma superfície de acordo com a direção da luz e a orientação do objeto. No entanto, ainda não eram programas independentes, mas estavam diretamente incorporados aos programas de render.

Em meados dos anos 80, a indústria do cinema começou a precisar de materiais muito mais variados: pele, metal, madeira, água, vidro, fumaça, entre outros. Os renderizadores da época já incluíam alguns materiais predefinidos, mas quando era necessário criar um novo, os desenvolvedores precisavam modificar o software. Isso fazia com que o sistema fosse pouco flexível e difícil de ampliar.

A Pixar propôs uma ideia diferente: em vez de o renderizador conhecer todos os materiais possíveis, ele deveria simplesmente ser capaz de executar pequenos programas escritos para descrever os materiais que fossem necessários em cada caso específico. Assim nasceu o **RenderMan**, apresentado em 1988, um dos primeiros sistemas a incorporar uma linguagem de shaders, que teve enorme influência no desenvolvimento posterior da computação gráfica. Naquele momento ainda não existiam as GPUs programáveis; todos os cálculos eram realizados na CPU.

Chegados os anos 90, a indústria dos videogames impulsionou o crescimento do 3D, deixando a CPU aquém para processar toda a informação gráfica. Para resolver esse problema, começaram a surgir os aceleradores gráficos que, com o tempo, evoluíram para o desenvolvimento das GPUs modernas.

Essas primeiras placas gráficas utilizavam uma **arquitetura de função fixa** (*Fixed Function Pipeline*). Isso significa que o hardware já vinha preparado para realizar determinadas tarefas gráficas, como transformar vértices, aplicar iluminação básica, texturas, névoa ou mesclas de cor (*blending*). No entanto, não era possível modificar a forma como esses

cálculos eram realizados; só era possível ativar, desativar ou configurar as funções que o fabricante havia incorporado na placa.

No final dos anos 90, a NVIDIA popularizou o termo **GPU** com a **GeForce 256** (1999), considerada pela própria empresa como a primeira Graphics Processing Unit. Pouco depois, com placas como a **GeForce 3** (2001), começaram a surgir comercialmente as primeiras GPUs com suporte a shaders programáveis.

Uma das mudanças mais importantes chegou no ano 2000 com o **DirectX 8**, um conjunto de bibliotecas desenvolvido pela Microsoft para criar jogos e aplicações multimídia no Windows. O DirectX 8 introduziu oficialmente os **Vertex Shaders** e os **Pixel Shaders**, permitindo pela primeira vez que se pudessem escrever pequenos programas para controlar como a GPU transforma os vértices e calcula a cor dos pixels. Depois disso, as novas gerações de placas gráficas começaram a incorporar essas capacidades.

Até esse momento, grande parte do aspecto final de uma imagem era determinada por funções fixas definidas pelo fabricante da placa gráfica. Com a chegada dos shaders programáveis, muitas dessas decisões passaram a ser escritas por meio de código. Em vez de se limitar a utilizar os efeitos disponíveis, tornou-se possível criar materiais, iluminações e efeitos visuais próprios e/ou personalizados.

Para que isso funcionasse, era necessária uma forma de comunicação dos programas com a placa gráfica.

Aqui aparece um conceito importante da computação: uma **API** (*Application Programming Interface*). Uma API pode ser entendida como um conjunto de ferramentas e regras que permite que um programa se comunique com outro, ou, nesse caso, com a placa gráfica, sem precisar conhecer como funciona internamente cada modelo.

Durante muitos anos existiram duas grandes APIs para a computação gráfica.

A primeira, **DirectX**, desenvolvida pela Microsoft principalmente para Windows, que utiliza a linguagem **HLSL** (*High Level Shader Language*).

A segunda, **OpenGL** (*Open Graphics Library*), um padrão aberto para programação gráfica que utiliza a linguagem **GLSL** (*OpenGL Shading Language*).

Para maior clareza, *OpenGL* é a API, o conjunto de ferramentas que permite que um shader programável se comunique com a placa gráfica. *GLSL* é a linguagem com a qual escrevemos o shader programável.

A OpenGL foi criada em 1992 pela empresa Silicon Graphics (SGI). Atualmente é mantida pelo **Khronos Group**, um consórcio internacional encarregado de desenvolver e manter diferentes padrões abertos para a computação gráfica.

O GLSL permite escrever shaders que podem ser utilizados em diferentes contextos. Diversos programas voltados para a arte digital e/ou a criação audiovisual permitem trabalhar com shaders escritos em GLSL, entre eles TouchDesigner, Resolume, Processing, openFrameworks, ou até mesmo podem ser integrados na web por meio do WebGL.

Também é um padrão multiplataforma, por isso os shaders escritos em GLSL podem ser utilizados no **Windows**, **Linux** e também no **macOS**. Embora a Apple atualmente desenvolva sua própria tecnologia gráfica, chamada **Metal**, ainda é possível trabalhar com GLSL por meio de aplicações compatíveis.

Com o passar dos anos, as GPUs continuaram evoluindo para oferecer cada vez mais capacidade de programação. Foram adicionadas novas etapas programáveis dentro do pipeline gráfico, como os **Geometry Shaders**, **Tessellation Shaders**, **Compute Shaders** e, mais recentemente, os **Mesh Shaders**. Ao mesmo tempo, surgiram novas técnicas de renderização, como **PBR** (*Physically Based Rendering*), **HDR** (*High Dynamic Range*), **Deferred Rendering** e/ou o **Ray Tracing** em tempo real.

Hoje em dia existem várias linguagens de shaders e/ou versões destas, entre elas **GLSL 1 a 4**, **HLSL**, **WGSL** (utilizado pela WebGPU) e **Metal Shading Language** (utilizado pela API Metal da Apple). Embora pertençam a ecossistemas diferentes, todas compartilham praticamente a mesma lógica. Mudam algumas palavras, a forma de declarar certos elementos e alguns aspectos próprios de cada plataforma, mas as ideias fundamentais são as mesmas. Às vezes uma variável pode ser declarada de uma forma, às vezes de outra, mas no fundo é a mesma coisa ou muito parecida.

Finalmente, para resumir o essencial, a ideia de shading evoluiu desde algoritmos matemáticos para calcular a cor de uma superfície até se converter em pequenos programas que são executados diretamente na GPU.

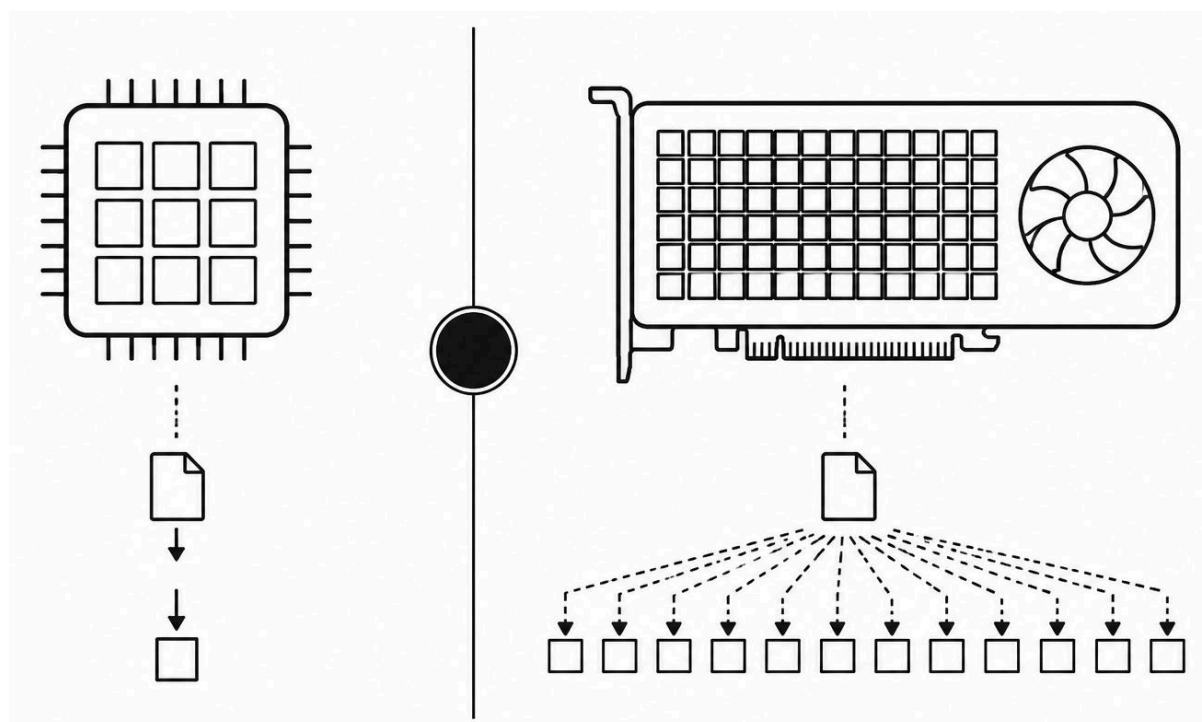
Referências:

- <https://openglbook.com/>
- <https://www.khronos.org/>
- <https://www.aswf.io/>

Como funcionam os shaders?

Os shaders são executados na GPU (Graphics Processing Unit), um processador especializado no cálculo de gráficos.

Ao contrário da CPU, a GPU conta com milhares de núcleos mais simples, capazes de executar o mesmo programa sobre uma grande quantidade de dados em paralelo. A CPU, por sua vez, dispõe de poucos núcleos, mas muito mais potentes e versáteis. Ela é projetada para executar programas de propósito geral, coordenar o funcionamento do sistema e resolver tarefas que exigem instruções diferentes e decisões complexas. Enquanto a CPU coordena o funcionamento geral da aplicação, a GPU realiza em paralelo os cálculos necessários para gerar a imagem.

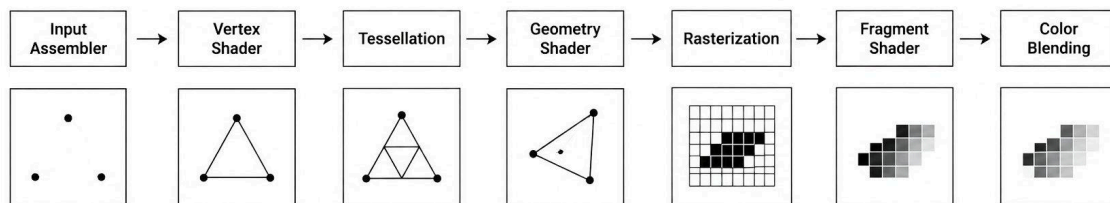


Os shaders se comunicam com a GPU por meio da API (*OpenGL*), que compila o código para que a GPU o execute. Eles enviam informações sobre a forma, os vértices, seu posicionamento no espaço e como estão conectados, além de informações sobre a cor, a textura, o material e a superfície que posteriormente será representada em pixels na tela.

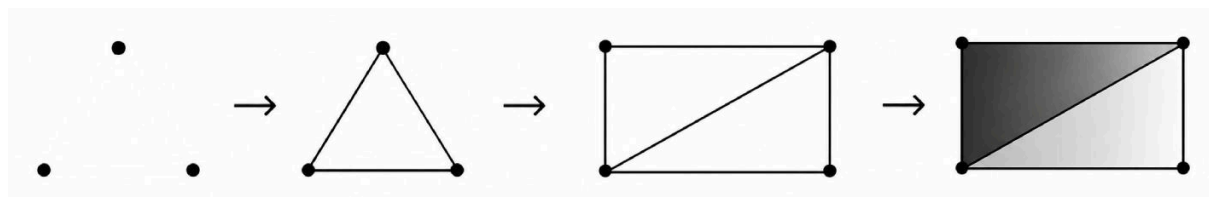
Para processar essa informação, o shader não é executado uma única vez, mas muitas vezes, seja porque parte da informação é processada de forma independente, seja porque para gerar cada frame é necessária uma nova execução. A GPU invoca o shader cada vez que precisa dele. Isso significa que o que se está vendo na tela é o que a GPU está processando naquele momento, e não uma informação guardada em um arquivo de vídeo, por exemplo. Nesse sentido, o shader não tem memória do que aconteceu antes nem pode ir adiante; trabalha unicamente com a informação disponível naquele momento. Se quisermos reaproveitar os frames para gerar um efeito, como um delay, podemos criar

buffers de memória, ou, se quisermos guardar o resultado como vídeo, é preciso gravar ou capturar os fotogramas com outro processo ou aplicação.

O processo de desenho de cada frame está dividido em várias etapas, conhecidas em seu conjunto como o *pipeline* gráfico. Nos sistemas gráficos modernos existem diferentes tipos de shaders que participam desse pipeline, como os *geometry shaders*, *tessellation shaders* ou *compute shaders*. No entanto, nós vamos trabalhar principalmente com dois tipos de shaders: o **vertex shader** e o **fragment** ou **pixel shader**. Todos os programas funcionam, no mínimo, com esses dois shaders e muitas vezes nem sequer existe a opção de utilizar os outros. Então, em um sentido prático, esses são os dois tipos de shaders que nos interessam.



O **vertex shader** é o que lida com a informação do espaço: as coordenadas e a posição dos vértices que vão formar a geometria sobre a qual será desenhada a superfície. As formas são construídas a partir de triângulos, que são a unidade básica com a qual a GPU constrói a geometria; sempre a união entre três pontos. Assim, para gerar uma superfície retangular clássica são necessários dois triângulos, e assim por diante, dependendo da complexidade da forma. Utilizam-se triângulos porque, com três pontos, tem-se a segurança de que sempre pertencem ao mesmo plano; com quatro ou mais pontos isso não necessariamente ocorre, o que poderia gerar erros. Os modelos 3D são formados por listas de vértices e pela informação de como estão conectados entre si. Essa informação é passada ao vertex shader para poder aplicar posteriormente um **fragment** ou pixel shader ao modelo 3D. O vertex shader responde a: **onde fica cada vértice?**



Uma vez definida a geometria, ela passa para o **fragment** ou **pixel shader**, que contém a informação sobre a cor e a textura dessa superfície. Chama-se fragment porque um fragmento é a informação que é gerada a partir do processamento do shader, enquanto um pixel é o resultado final que aparece na tela. Às vezes um fragmento se reflete em um pixel e outras vezes não, porque pode haver transparência ou ele pode ficar oculto dentro da imagem. Por isso, o nome tecnicamente correto é fragment shader, embora muita gente também o conheça como pixel shader. O fragment shader responde a: **de que cor será cada pixel?**

A geração de imagens por meio de shaders se baseia principalmente na álgebra linear. Ela permite representar, posicionar e transformar a geometria no espaço, assim como realizar operações sobre a cor e outros atributos visuais, por meio de vetores, matrizes e funções

matemáticas, como as trigonométricas. Não é preciso ser um especialista, mas é bom saber disso se quisermos entender um pouco mais sobre como a imagem é gerada.

O principal a ter claro são o **vertex shader** e o **fragment shader**, que são os scripts, ou arquivos de texto, nos quais vamos escrever o programa que será executado. O vertex shader se encarrega principalmente da geometria, embora também possa lidar com informação de cor, e o fragment shader calcula o valor final dos pixels. Ambos trabalham de maneira coordenada para gerar cada imagem. Então, sempre vamos precisar de dois scripts diferentes, o vertex e o fragment; embora haja casos em que o software só permite modificar o fragment shader, porque o vertex shader já vem definido por padrão. Esse é o caso do *Resolume*, por exemplo. O *Resolume* assume que o resultado final sempre será o retângulo de saída do layer onde o shader é executado e, por essa razão, já vem com o vertex shader retangular pré-estabelecido, permitindo carregar apenas o fragment shader.

O texto pode conter algumas imprecisões técnicas avançadas em prol da intuição.

ISF (Interactive Shader Format)

O **ISF (Interactive Shader Format)** é um **formato** criado para que um mesmo shader pudesse rodar em programas diferentes sem precisar ser reescrito para cada um. A ideia foi padronizar uma forma de escrever shaders para que funcionassem em vários programas sem necessidade de adaptação, na maioria dos casos. Esse formato foi desenvolvido em 2013 pela **Vidvox**, desenvolvedora do software *Vdmx*, pensado originalmente para esse programa, e depois foi adotado por outros.

Hoje em dia o formato é compatível com softwares como *Resolume* (via *Wire*), *MadMapper*, *Smode*, *Millumin* entre outros. A *Vidvox* também desenvolveu um editor e visualizador prático e multiplataforma chamado **ISF Editor**, oferecendo uma forma simples de testar e ajustar shaders ISF.

Um shader ISF continua sendo, em essência, um shader GLSL, mas com alguns ajustes, como um bloco em formato JSON, no qual é feita uma descrição e são declarados os inputs que queremos que fiquem disponíveis para manipular a imagem, além de algumas variáveis já definidas automaticamente (*built-in*) e certas convenções próprias do formato.

Uma das vantagens de declarar os inputs no bloco JSON é que eles ficam acessíveis diretamente na interface do programa como sliders, botões, pads ou outros controles, dependendo do tipo de input que tenha sido definido.

Graças às tecnologias de IA de hoje em dia, é bem simples converter um shader clássico GLSL em um ISF.

Referências

- <https://isf.vidvox.net/desktop-editor/>
- <https://isf.video/>
- <https://editor.isf.video/>
- <https://www.vidvox.net/>

Vertex Shader

Nesta seção, revisaremos dois **vertex shaders** escritos em **GLSL ES 1.00** (WebGL 1 / OpenGL ES 2.0). À medida que os analisamos, iremos explicando os conceitos fundamentais da linguagem.

```
src > shaders > ≡ vertex.glsl
1  uniform mat4 projectionMatrix;
2  uniform mat4 viewMatrix;
3  uniform mat4 modelMatrix;
4
5  attribute vec3 position;
6  attribute vec2 uv;
7
8  varying vec2 v_uv;
9
10 void main() {
11     v_uv = uv;
12
13     gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.);
14 }
15
```

uniform mat4 projectionMatrix;

uniform mat4 viewMatrix;

uniform mat4 modelMatrix;

uniform : os **uniforms** são variáveis cujo valor é enviado do programa para o shader e permanece igual para todos os vértices ou fragmentos enquanto um objeto é desenhado.

mat4 : é o tipo de dado, uma **matriz de 4 × 4**, uma tabela de **4 linhas e 4 colunas** de números, ou 4 **vec4**

Tipos de dados: nos shaders, todos os dados têm um tipo definido para que a GPU possa realizar os cálculos matemáticos de forma eficiente. Como os shaders trabalham principalmente com geometria e gráficos, seus tipos de dados são pensados para a álgebra linear.

Os tipos mais comuns são:

- **int**: números inteiros.
- **float**: números decimais ou de ponto flutuante.
- **vec2**, **vec3** e **vec4**: vetores de dois, três e quatro componentes.
- **mat2**, **mat3** e **mat4**: matrizes de 2×2, 3×3 e 4×4.
- **bool**: valores booleanos, que só podem ser **true** ou **false** (verdadeiro ou falso).

Na prática, os **int** são pouco usados. A maior parte do tempo trabalha-se com **float**, já que quase todos os cálculos de posição, cor, movimento e iluminação exigem números

decimais. Os **vetores** também são um dos tipos de dados mais utilizados, porque permitem representar posições, direções, cores e coordenadas de textura.

As operações matemáticas só podem ser realizadas entre tipos de dados compatíveis. Se os tipos não coincidirem, o shader gerará um erro de compilação. Por exemplo, não é possível realizar diretamente uma operação entre um **int** e um **float**:

```
int x = 5;  
float y = 2.5;  
float z = x + y; // Error
```

Se for necessário combinar diferentes tipos de dados, primeiro é preciso converter um deles para o tipo adequado. Isso é conhecido como conversão de tipo ou casting.

```
float z = float(x) + y;  
ou então:  
int z = x + int(y);
```

Os vetores (**vec2**, **vec3**, **vec4**) e as matrizes (**mat2**, **mat3**, **mat4**) também podem participar de operações matemáticas, desde que os tipos sejam compatíveis. Na prática, quase sempre contêm valores do tipo float, por isso é comum trabalhar com **float**, **vec** e **mat**.

; : nos shaders, todas as instruções devem terminar com um ;. Se for omitido, o shader gerará um erro de compilação. Esquecê-lo é um dos erros de sintaxe mais comuns. A exceção são as **diretivas do pré-processador** (por exemplo, #define ou #ifdef), que não utilizam ponto e vírgula.

uniform mat4 projectionMatrix; : declara a **matriz de projeção**, que transforma a cena para que ela possa ser vista a partir da câmera e projetada na tela (perspectiva ou ortográfica).

uniform mat4 viewMatrix; : declara a **matriz de visualização**, que representa a posição e orientação da câmera na cena.

uniform mat4 modelMatrix; : declara a **matriz de modelo**, que contém a posição, rotação e escala do objeto no mundo.

Embora às vezes seja necessário declará-las como variáveis, essas matrizes normalmente já vêm predefinidas pelo motor gráfico, portanto não é necessário criá-las nem atribuir-lhes um valor manual prévio.

```
5  attribute vec3 position;  
6  attribute vec2 uv;  
7  
8  varying vec2 v_uv;
```

attribute vec3 position;
attribute vec2 uv;

attribute : os **attributes** são variáveis que armazenam dados pertencentes a cada vértice da geometria. Diferentemente dos **uniforms**, cujo valor é o mesmo para todos os vértices, os **attributes** podem ter um valor diferente em cada um. Esses dados podem representar a posição de cada vértice, suas *coordenadas*, suas *normais* (o vetor perpendicular à superfície, utilizado principalmente para os cálculos de iluminação em 3D) ou até mesmo uma *cor* específica para cada vértice.

attribute vec3 position; : **position** contém a posição de cada vértice no espaço local do objeto. É um **vec3** porque cada posição é formada por três componentes: **x**, **y** e **z**, correspondentes ao espaço tridimensional.

attribute vec2 uv; : **uv** contém as coordenadas de textura de cada vértice. É um **vec2** porque é formado por dois componentes: **u** e **v**, que correspondem a um espaço bidimensional utilizado para mapear uma textura sobre a geometria.

varying vec2 v_uv;

varying : os **varying** são variáveis que permitem enviar informações do **vertex shader** para o **fragment shader**. Normalmente é informação espacial ou de coordenadas.

varying vec2 v_uv; : declara-se a variável **v_uv**, que será utilizada para enviar as coordenadas (uv) do vertex shader para o fragment shader. É um **vec2** porque deve corresponder ao tipo de dado que vai enviar.

Por convenção, costuma-se utilizar o prefixo **a** para os **attributes**, **u** para os **uniforms** e **v** para os **varying**. Embora não seja obrigatório e nem sempre você encontre essa nomenclatura, é uma boa prática, pois facilita identificar o tipo de cada variável ao ler o código.

```

10 void main() {
11     v_uv = uv;
12
13     gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.);
14 }

```

void main() : é a função principal do shader, onde são escritas todas as instruções que a GPU deve executar. Em um vertex shader, essa função é executada **uma vez para cada vértice** da geometria.

v_uv = uv; : aqui, o valor do atributo uv é copiado para o **varying v_uv**, para enviá-lo ao *fragment shader*.

gl_Position : é a variável mais importante do vertex shader, é o resultado final do vertex shader. Nela é escrita a posição final de cada vértice.

gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4(position, 1.); : esta é a forma clássica de calcular a posição final de um vértice. Primeiro, **position** é convertido de um **vec3** para um **vec4**, já que as matrizes utilizadas são do tipo **mat4** e ambos os tipos precisam ser compatíveis para poderem ser multiplicados. Em seguida, as matrizes **modelMatrix**, **viewMatrix** e **projectionMatrix** transformam essa posição do espaço local do objeto até sua posição final na tela.

projectionMatrix : projeta a cena sobre a tela.

viewMatrix : aplica a posição e orientação da câmera.

modelMatrix : posiciona o objeto na cena.

```

shaders > ≡ vertex.glsl
1  attribute vec3 a_position;
2  varying vec2 v_texcoord;
3
4  void main(void) {
5      v_texcoord = a_position.xy;
6
7      gl_Position = vec4(a_position, 1.0);
8  }

```

Este é outro vertex shader. É muito parecido com o anterior, mas não utiliza as matrizes **modelMatrix**, **viewMatrix** nem **projectionMatrix**. Isso ocorre porque as posições armazenadas em **a_position** já vêm preparadas para serem escritas diretamente em **gl_Position**, portanto não precisam de transformações adicionais. Dependendo do motor gráfico ou do tipo de aplicação, essas transformações podem ser realizadas antes de os dados chegarem ao shader.

v_texcoord = a_position.xy; : aqui, apenas os componentes **x** e **y** de **a_position** são copiados e armazenados em **v_texcoord**.

a_position é um **vec3**, portanto contém três componentes: **x**, **y** e **z**. No entanto, **v_texcoord** é um **vec2**, portanto só pode armazenar **dois** componentes. A notação **.xy** indica que queremos utilizar apenas esses dois valores.

Essa técnica é conhecida como *swizzling*, e permite acessar ou reorganizar os componentes de um vetor. Por exemplo:

- `position.x` → obtém apenas o componente **x**.
- `position.xy` → obtém os componentes **x** e **y** como um **vec2**.
- `position.xyz` → obtém os três componentes como um **vec3**.
- `position.yx` → obtém os componentes **y** e **x**, invertendo sua ordem.

Nos shaders, os vetores podem ter até **quatro** componentes. Para acessá-los, utilizam-se três nomenclaturas equivalentes:

x, y, z, w : usados normalmente para posições e coordenadas espaciais.

r, g, b, a : usados habitualmente para representar cores.

s, t, p, q : utilizados principalmente para coordenadas de textura.

As três nomenclaturas referem-se aos mesmos componentes; muda apenas o nome, para que o código fique mais fácil de ler conforme o contexto.

Por exemplo, se tivermos: **vec4** dato;

Podemos acessar seus componentes de diferentes formas:

`dato.xy`

`dato.rgb`

`dato.st`

`dato.yx`

`dato.bgra`

Essa técnica é conhecida como *swizzling* e permite selecionar ou reordenar os componentes de um vetor. A única coisa que não é permitida é misturar nomenclaturas em um mesmo acesso. Por exemplo, `dato.xg` produziria um erro.

Fragment Shader

Nesta seção, revisaremos dois **fragment shaders** escritos em **GLSL ES 1.00** (WebGL 1 / OpenGL ES 2.0). À medida que os analisamos, iremos explicando os conceitos fundamentais da linguagem.

```
src > shaders > fragment.glsl
1  precision mediump float;
2
3  uniform float u_time;
4
5  varying vec2 v_uv;
6
7  void main() {
8
9      float t = sin(u_time) * 0.5 + 0.5;
10
11
12      gl_FragColor = vec4(v_uv, t, 1.);
13  }
14
```

precision mediump float; : esta linha define a **precisão padrão** que os valores do tipo **float** terão no fragment shader. Existem três níveis de precisão: **lowp**, **mediump** e **highp**, que representam uma precisão baixa, média e alta, respectivamente. Há ocasiões em que não é necessário colocá-la.

uniform float u_time; : é uma variável que armazena o tempo decorrido.

Vale lembrar que os **uniforms** são variáveis cujos valores são enviados a partir do ambiente (JavaScript, Three.js, OpenGL, etc.). No fragment shader, é muito comum utilizar **uniforms** para enviar informações como o tempo (**u_time**), texturas (imagens, vídeos ou a câmera), a resolução da tela, a posição do mouse ou qualquer outro valor que queiramos controlar a partir do programa — variáveis que podemos ter como sliders, botões ou sensores em nossa interface, por exemplo.

varying vec2 v_uv; : esta linha declara a variável **v_uv**, que recebe a informação enviada pelo vertex shader. Neste caso, as coordenadas **uv**.

void main() : assim como no vertex shader, é a função principal do shader. Todo o código que está dentro dela é executado pela GPU. No fragment shader, essa função é executada **uma vez para cada fragmento**. Enquanto o vertex shader processa os vértices da geometria, o fragment shader processa os **fragmentos**, que posteriormente se transformarão nos **pixels** visíveis da imagem.

float t = sin(u_time) * 0.5 + 0.5; : declara uma variável chamada **t** e atribui a ela um valor que muda continuamente com o tempo.

- **sin(u_time)** : calcula o seno do tempo. Esse valor oscila continuamente entre -1. e 1., gerando um movimento periódico.
- *** 0.5** : reduz essa amplitude pela metade, fazendo com que o intervalo passe de -1. a 1., para -0.5 a 0.5
- **+ 0.5** : desloca o intervalo para cima, obtendo finalmente valores entre 0. e 1.

Como resultado, **t** aumenta e diminui suavemente entre 0. e 1. à medida que o tempo.

Nos shaders, é muito comum **normalizar** valores, ou seja, transformá-los para que fiquem dentro do intervalo de 0 a 1. Muitos parâmetros, como as cores, a transparência, as misturas ou as coordenadas UV, utilizam essa faixa. Esse intervalo entre **0** e **1** é conhecido como **intervalo normalizado**.

Trabalhar com valores normalizados torna os cálculos mais consistentes e independentes da resolução ou do tamanho da tela.

Por exemplo, é comum utilizar um sistema de **coordenadas normalizadas**, de **0.0** a **1.0**. É possível trabalhar diretamente com coordenadas em pixels, mas isso costuma fazer com que o shader dependa da resolução. Se o tamanho da janela mudar, é necessário adaptar os cálculos. Já as **coordenadas normalizadas** se ajustam automaticamente a qualquer resolução e facilitam o desenvolvimento de shaders reutilizáveis.

As cores também são representadas com valores normalizados. Cada canal do modelo **RGBA (r, g, b, a)** assume um valor entre **0.0** e **1.0**, onde **0.0** significa ausência desse componente e **1.0** sua intensidade máxima. No **canal alfa**, **0.0** representa uma transparência total e **1.0** uma opacidade completa.

Nos shaders, quando escrevemos um valor do tipo **float**, o número deve incluir um ponto decimal. Por exemplo:

1.0
1.
0.7
.7

Todas essas formas representam um **float**. Não é obrigatório escrever o 0 antes ou depois do ponto quando essa parte vale zero, mas é obrigatório escrever o ponto decimal.

Se escrevermos:

1

O programa interpreta esse valor como um **int**, não como um **float**. Isso pode gerar erros ao realizar operações entre tipos de dados incompatíveis, *esquecer o ponto decimal é um dos erros mais comuns ao começar a programar shaders*.

gl_FragColor : é a variável principal do fragment shader, assim como o **gl_Position** é para o *vertex shader*. Nela é escrito a **cor final** de cada **fragmento**.

gl_FragColor = vec4(v_uv, t, 1.); : como **gl_FragColor** é um **vec4**, devemos construir um vetor com quatro componentes. Neste caso:

- **v_uv** é um **vec2**, portanto fornece os dois primeiros componentes do vetor.
- **t** é um float e fornece o terceiro componente.
- 1. fornece o quarto componente.

É equivalente a escrever: `gl_FragColor = vec4(v_uv.x, v_uv.y, t, 1.);`

Como **gl_FragColor** representa uma **cor**, esses **quatro** componentes correspondem aos canais **rgba**: **vermelho** (red), **verde** (green), **azul** (blue) e **alfa** (transparência), respectivamente

Ver resultado deste fragment shader : <https://santirockk.github.io/fsexample1/>

Agora vamos ver um fragment shader um pouco mais complexo. Não é necessário compreender em detalhe como ele funciona. Por enquanto, o importante é observar sua estrutura e ter uma ideia de como se constrói um fragment shader mais elaborado.

```
precision mediump float;

uniform float u_time;
uniform vec2 u_resolution;
varying vec2 v_texcoord;

#define PI 3.14159265359
#define TAU (PI * 2.)

vec3 blue = vec3(0.18, 0.7, 0.92);
vec3 orange = vec3(0.87, 0.49, 0.38);
vec3 green = vec3(0.3, 0.65, 0.43);
vec3 peach = vec3(0.96, 0.76, 0.55);
vec3 black = vec3(0.);

float POINTS = 24.;
float speed = 0.15;
float dotRadiusMask = 0.19;
float scale = 0.77;
float radius_a = 0.23 ;

// Shapes
float circle(vec2 p, float r) {
    return 1. - smoothstep(r, r + 0.003, length(p));
}

float ringMask(float d, float r, float w) {
    float outer = smoothstep(r + w * .5, r + w * .5 - 0.002, d);
    float inner = smoothstep(r - w * .5, r - w * .5 + 0.002, d);
    return outer * inner;
}
```



```

float dotMask(vec2 uv, float r, float w, float direction) {
    float d = length(uv);
    float angle = atan(uv.y, uv.x);
    angle += direction * u_time * speed;
    float sector = TAU/POINTS;
    float local = mod(angle + sector * .5, sector) - sector * .5;

    vec2 p;
    p.x = d - r;
    p.y = local * r;
    float dotRadius = w * dotRadiusMask;
    return circle(p, dotRadius);
}

void drawRing(
    inout vec3 color,
    vec2 uv,
    float radius,
    float width,
    vec3 ringColor,
    vec3 dotColor,
    float direction
) {
    float d = length(uv);
    float ring = ringMask(d, radius, width);
    if(ring < 0.001) return;
    color = mix(color, ringColor, ring);
    float dots = dotMask(uv, radius, width, direction);
    color = mix(color, dotColor, dots * ring);
}

void main() {
    vec2 uv = v_texcoord;
    uv.x *= u_resolution.x/u_resolution.y;

    vec3 color = blue;
    float radius = 2.;

    for(int i=0; i<21; i++) {
        bool org = mod(float(i), 2.) == 0.;

        drawRing(color,
            uv,
            radius,

```

```

        radius * radius_a,
        org ? orange : green,
        org ? black : peach,
        org ? 1. : -1.
    );
    radius *= scale;
}

gl_FragColor = vec4(color, 1.);
}

```

uniform float u_time;

uniform vec2 u_resolution;

varying vec2 v_texcoord;

uniform float u_time; a variável que armazena o tempo decorrido

uniform vec2 u_resolution; essa variável armazena a resolução da janela de saída, o *width* e o *height*, em um vetor 2; às vezes pode ser útil para certos cálculos.

varying vec2 v_texcoord; variável que recebe a informação de *a_position.xy* do vertex shader.

#define PI 3.14159265359

*#define TAU (PI * 2.)*

Variáveis para o pré-processador não levam ";" no final.

vec3 blue = vec3(0.18, 0.7, 0.92);

vec3 orange = vec3(0.87, 0.49, 0.38);

vec3 green = vec3(0.3, 0.65, 0.43);

vec3 peach = vec3(0.96, 0.76, 0.55);

vec3 black = vec3(0.);

Paleta de cor, não necessariamente deve-se declarar dessa forma, mas eu gosto de fazer assim.

vec3 black = vec3(0.); se, por exemplo, todos os 3 valores desse vetor forem iguais, neste caso 0., não é necessário declarar *vec3(0., 0., 0.);* ao declarar *vec3(0.)*, o programa assume que todos os componentes do *vec3* são 0.

float POINTS = 24.;

float speed = 0.15;

float dotRadiusMask = 0.19;

float scale = 0.77;

float radius_a = 0.23 ;

Variáveis que deixamos livres para intervir, modificar, ajustar o resultado final.

Eventualmente, podemos transformá-las em *uniforms* e depois em sliders para controlar seus valores a partir da interface.

```

// Shapes
float circle(vec2 p, float r) {
    return 1. - smoothstep(r, r + 0.003, length(p));
}

float ringMask(float d, float r, float w) {
    float outer = smoothstep(r + w * .5, r + w * .5 - 0.002, d);
    float inner = smoothstep(r - w * .5, r - w * .5 + 0.002, d);
    return outer * inner;
}

float dotMask(vec2 uv, float r, float w, float direction) {
    float d = length(uv);
    float angle = atan(uv.y, uv.x);
    angle += direction * u_time * speed;
    float sector = TAU/POINTS;
    float local = mod(angle + sector * .5, sector) - sector * .5;
    vec2 p;
    p.x = d - r;
    p.y = local * r;
    float dotRadius = w * dotRadiusMask;
    return circle(p, dotRadius);
}

void drawRing(
    inout vec3 color,
    vec2 uv,
    float radius,
    float width,
    vec3 ringColor,
    vec3 dotColor,
    float direction
) {
    float d = length(uv);
    float ring = ringMask(d, radius, width);

    if(ring < 0.001) return;
    color = mix(color, ringColor, ring);
    float dots = dotMask(uv, radius, width, direction);
    color = mix(color, dotColor, dots * ring);
}

```

Funções (*auxiliares*) que vão nos ajudar a criar a forma ou as formas que queremos para nossa imagem final. Como podem notar, a terceira função **dotMask()** utiliza a primeira função **circle()**, e a quarta função **drawRing()** utiliza a segunda e a terceira função, **ringMask()** e **dotMask()**; em seguida, essa quarta função **drawRing()** será a que chamaremos na função **main()**.

É muito comum construir funções a partir de outras funções. Dessa forma, a partir de operações simples é possível criar formas e comportamentos cada vez mais complexos. Esse princípio é uma das bases da programação e também do desenvolvimento de shaders.

Também podemos notar que essas funções estão declaradas fora de **main()**. Isso ocorre porque **main()** é a função principal do shader, enquanto as demais são **funções auxiliares** que podem ser chamadas a partir de **main()** ou até mesmo de outras funções. Declará-las separadamente torna o código mais claro, mais fácil de reutilizar e mais simples de manter.

Se observarmos com um pouco mais de detalhe o interior dessas funções, encontraremos chamadas a outras funções como **smoothstep()**, **length()**, **atan()**, **mod()** ou **mix()**. Elas fazem parte da biblioteca padrão do GLSL, portanto já vêm incluídas na linguagem e não é necessário declará-las.

Para saber quais funções estão disponíveis e como funcionam, recomendamos consultar o glossário do The Book of Shaders, uma excelente referência para aprender GLSL:
<https://thebookofshaders.com/glossary/>

Neste exemplo, também encontramos estruturas comuns de programação, como os loops **for** :

```
for(int i = 0; i < 21; i++)
```

estruturas condicionais do tipo:

```
if(ring < 0.001) return;
```

ou **operador condicional (ternário)**:

```
org ? orange : green
```

A ideia não é aprofundar nesses conceitos, já que pertencem à programação em geral, mas sim mostrar que esse tipo de técnica também é utilizado dentro dos shaders.

uv.x *= u_resolution.x / u_resolution.y; : esta linha nem sempre é necessária. É utilizada para corrigir a relação de aspecto da tela, de modo que uma unidade no eixo **X** tenha o mesmo tamanho visual que uma unidade no eixo **Y**. Isso evita que as formas se deformem quando a janela não é quadrada; por exemplo, permite que um círculo continue parecendo um círculo, e não uma oval.

Ver resultado deste fragment shader : <https://santirockk.github.io/fspoints/>

Referências:

- <https://thebookofshaders.com/>

Referências:

- <https://thebookofshaders.com/>
- <https://www.shadertoy.com/>
- <https://openglbook.com/>
- <https://www.khronos.org/>
- <https://isf.video/>
- <https://www.vidvox.net/>
- <https://www.aswf.io/>
- <https://www.shadertoy.com/>
- <https://isf.vidvox.net/desktop-editor/>
- <https://editor.isf.video/>

Por AV8, Agosto 2026

<https://av8visual.vercel.app/>